

Riva Snap (Redesigned)

User Persona: Sarah, 32 yrs, Working, Health Conscious

She has a deep bowl packed with brown rice, grilled chicken, black beans, and a generous scoop of guacamole. She wants to track her nutrition accurately.

Core Pain Points and Key Needs

1. The Invisible Ingredients Problem

Sarah knows exactly how her food was prepared, but she knows a camera lens cannot see it. A single tablespoon of invisible olive oil, a pat of butter used to sear chicken, or heavy cream mixed into a sauce can easily add over 100 hidden calories. None of this shows up in a flat photograph.

- **Key Need:** A quick way to tell the app about cooking oils, hidden condiments, or underlying marinades without having to dig through deep search menus.

2. 2D Images Hide Food Volumetrics

A flat image completely hides the vertical height of a food pile. A plate with a thin layer of rice looks almost identical from a top-down view to a plate stacked with a massive, dense mound of rice.

- **Key Need:** A continuous way to capture food that calculates vertical volume, pile height, and container depth. This ensures the app can tell a light snack apart from a heavy meal portion.

3. Identical Looking Foods Confusion

2D images lack physical depth details, AI models frequently misclassify foods that look identical from above. They easily mistake a bowl of Greek yogurt for sour cream, or mashed potatoes for cauliflower purée.

- **Key Need:** A simple way to anchor the AI's focus onto the exact dish on her plate. Combining her input with the camera's focus lets the system skip the guesswork and get the food item right the first time.

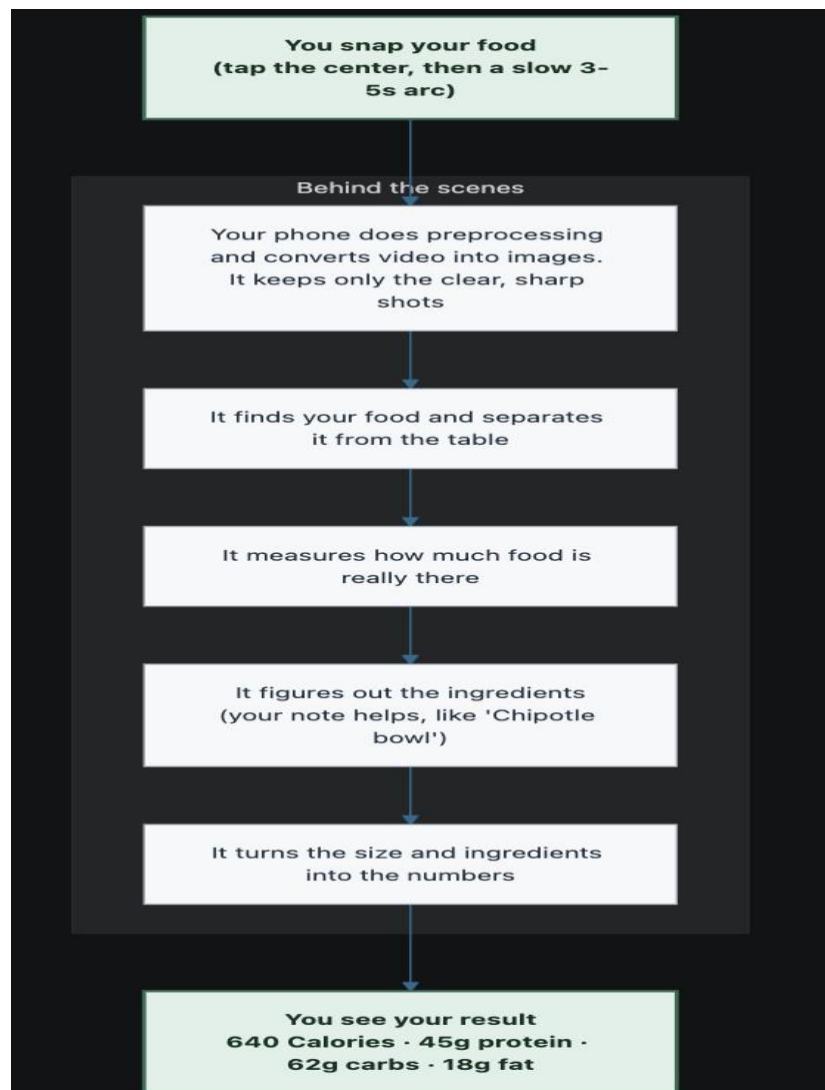
4. Multi-Angle Photo Requirements for correct estimation

Traditional tracking apps force users to take multiple separate photos, one photo from 45 degree angle, other from top and so on. This process takes up to **20 seconds** of clicking, moving around, and verifying images. Doing this at a crowded table or in a restaurant feels embarrassing.

- **Key Needs:** A fast, natural interaction that takes less than five seconds. It should capture multiple visual angles in one smooth motion without forcing her to stand up or take multiple photos.

Proposed Solution

Agentic AI based Tap and Hold scan - (Riva Scan)



User Journey

Camera and video access is requested once, up front, during onboarding.

Step 1: User Intent

Sarah sits at her desk and holds her phone naturally over her lunch. The screen clearly instructs her: **"Tap the center of your dish to lock."** She taps right on the middle of the chicken pile. A sharp, green target icon locks onto that selected item on her screen and tracks it.

- **The Mechanics:** The moment Sarah taps, the app captures the raw pixel touchpoint and immediately normalizes it into fractional coordinates between 0.0 and 1.0 (for example, X: 0.54, Y: 0.49).

By forcing Sarah to tap the center before she starts moving the camera, we anchor the entire pipeline. The backend no longer has to blindly guess what to track. It ignores the background and tracks the food. The tap anchors the whole dish, not just the spot she touched; SAM 2 then segments every item in it.

Step 2: Tap and Record

The UI text changes: **"Hold the button and scan slowly in an arc."** Sarah presses her thumb down on the record button. She slowly moves her phone in a smooth 3-5 second semi-circle arc, tilting the phone from a 45-degree side profile up to a top-down angle. She releases her thumb, and the camera viewfinder closes with an option to retake or proceed.

- **The Mechanics:** The app records a brief 3-5 seconds .mp4 video clip. Because Sarah moves her phone, the camera views the bowl from multiple changing angles. This physical movement creates **parallax**, where objects closer to the camera lens appear to shift position faster than objects further away in the background.

This parallax motion can be used to create a "structure-from-motion" effect. The video stream captures the height, width and curves of the food pile that a flat 2D image completely flattens out.

Step 3: Preprocessing

Sarah passes a clean loading screen that says: *"Target Locked. Processing your food..."* She doesn't realize it, but her phone is doing optimisations for faster pre processing.

- **The Mechanics:** The app compresses the video to an optimized 720p format. A native Swift framework instantly extracts 15 evenly spaced frames from the clip and runs a mathematical

edge-sharpness test on each one. Sarah's hand may shake slightly during the second second of the scan, creating motion blur. The app detects this, drops those blurry frames, and seamlessly grabs the sharpest adjacent frames instead. It selects the best 5-6 images....It then bundles the selected 5-6 images (never the raw video), the (X, Y) tap coordinates, and her optional text hint ("Chipotle chicken bowl") into a single network payload.

Uploading only the selected images, not the raw video, saves battery, bandwidth and user's cellular or wifi data, while Blurry images ruin depth estimation. Filtering out motion blur on the device ensures the cloud server only receives high-fidelity data.

Step 4: Cloud based Models

The loading text updates on Sarah's screen: "A Bowl filled with chicken and vegetables Detected ! Analyzing ingredient volume..."

Miles away on a high-speed GPU cloud server, the payload is unwrapped via a FastAPI endpoint. A scheduled job pings the service every five minutes, so the instance never cold-starts and Sarah's first request is answered immediately.

- **The Mechanics:** An Orchestration Agent acts as a router and decides what tools to use based on the user's hint and selected images. It reads Sarah's text hint: "Chipotle chicken bowl". The agent runs a parallel pipeline. It loads a specific JSON preset from the prebuilt standard recipe library for a standard Chipotle container (Diameter: 20.3 cm) for reference only. Simultaneously, the server splits the visual work into three parallel tracks:
 1. **SAM 2 (The Tracker):** Meta's Segment Anything 2 receives the original tap coordinates. It locks onto the whole dish (the bowl and everything in it) in Frame 1 and utilizes its streaming memory to track the moving food mask perfectly across all selected frames, despite Sarah's camera movement.
 2. **YOLOv11 (The Scale Anchor):** If Sarah hadn't typed "Chipotle", YOLOv11 would actively scan the background to find a standard reference object (like a fork or a pen) to calculate real-world measurements for better volumetric analysis.
 3. **Depth Anything V2 (The Depth Map):** This model generates a dense, relative depth heatmap of the selected images, turning the pixels into a grid of distances by the pixel color.

Running these heavy vision models in a parallel split-second architecture on the cloud ensures Sarah isn't left waiting. While she is waiting, the interface shows her "Did you know" facts about food for her. The total processing turnaround happens in under 4-5 seconds.

Step 5: The Math of Volumetric Space

While the scanning animation plays on Sarah's phone, the server math engine merges the AI outputs to calculate physical space.

- **The Mechanics:** The engine overlays the SAM 2 moving food mask directly onto the Depth Anything 2 map, instantly cutting away the desk and background. Because it knows the exact physical size of the bowl (either from the Chipotle preset or a fork scaled by YOLOv11), it establishes a metric multiplier (e.g., **1 pixel = 0.05 cm**). The engine measures the relative depth from the rim of the bowl to the top peak of the chicken pile. It integrates thousands of microscopic pixel columns to compute the estimated physical volume: about **580 cubic centimeters (mL)** of total food mass.

By using relative monocular depth maps combined with a known physical scale anchor, we get true volumetric accuracy without the slow, computationally expensive overkill of rendering a full 3D mesh.

Two depth strategies, chosen by the device: Riva Scan reads the phone's capabilities and picks the best available path.

- **ARKit devices (preferred):** On any iPhone (or Android with ARCore) that exposes motion tracking, we capture the camera's real-world pose at every frame of the arc. Because ARKit reports where the lens is, in meters, at each moment, the selected images 5-6 selected images from the video become a calibrated multi-view set. Triangulating the food surface across those views rebuilds true metric structure, giving us real scale with no reference object needed.
- **Default devices (fallback):** If the phone has no motion tracking, we lean on Depth Anything V2's monocular depth map and anchor it to the real world using the container preset or a YOLOv11-detected reference object. The extra angles sharpen identification, while the preset or reference fixes the scale. This baseline path runs on every phone.

Step 6: Final Conversion

A beautiful, highly detailed nutrition summary panel slides up smoothly on Sarah's screen. It reads: 🍽️ **640 Calories 45g Protein, 62g Carbs, 18g Fat** and gets into a human loop for approval. Sarah approves, adjusts her napkin, completely satisfied, and digs into her lunch.

- **The Mechanics:** A Multimodal LLM looks at the crispest frame alongside Sarah's text hint to determine the ingredient distribution (e.g., 50% rice, 25% chicken, 15% beans, 10% guacamole). The math engine multiplies the 580 mL volume by these ratios, applies the physical mass densities from the custom standard database of metrics per cubic centimetres recipes (converting volume directly to grams), and pulls the macronutrient metrics from the USDA database to display the result.

The text hint solves the fatal flaw of visual food trackers, invisible ingredients. It accounts for hidden oils, heavy seasoning, or ingredients buried at the bottom of the bowl, delivering an accurate calculation without forcing Sarah to live her life tethered to a digital kitchen scale.

Because each stage (the volume, the ingredient ratios, the densities, and the USDA lookup) carries its own margin of error, those errors can stack up. That is why the final number passes through Sarah's one-tap approval, where she can nudge a portion or correct an ingredient before it is logged.

Edge Cases and Fallbacks

- **No reference object found:** Not a blocker. On ARKit devices the scale comes from camera motion, and on default devices the container preset supplies it. The reference object is only a secondary anchor, so its absence never stops a scan.
- **Blurry or shaky video:** The app asks Sarah to retake, suggests wiping the lens, and gives live, dynamic guidance on how to record: slow the arc, keep a steady hand, add a little light.
- **No food detected:** The app says "No food detected," briefly names what it did see, and suggests recording one short clip per single item.

Evaluation

We benchmark accuracy against Nutrition5k dataset. Its images are captured top-down only, so a synthetic data generator will regenerate alternate viewing profiles, angled and side views, of each dish, producing the multi-view test set we need to measure volume and nutrition accuracy end to end.